ENGINEERING DECISION · ASYNCHRONOUS WORK

A queue is not extra capacity

Accept a burst without promising an endless wait. Choose a queue limit, a recovery budget, and an honest response when useful work no longer fits.

Fictional design review · illustrative assumptions, not production measurements

THE ARGUMENT

1. Capacity and waiting
2. The hidden cost of acceptance
3. A bounded promise

CAPACITY AND WAITING

A fast acknowledgment can hide a slow service

Accepted is a promise to finish, not evidence of completion.

A fictional report service moves generation behind a queue. Requests return quickly, but the workers do the same work at the same rate. The waiting has moved out of the request; it has not disappeared.

> ASSUMPTIONS
>
> Reports can finish asynchronously, but not indefinitely late. Jobs have comparable cost, workers share a constrained database, and the initial queue is empty. Numerical inputs below are invented for a simplified model.

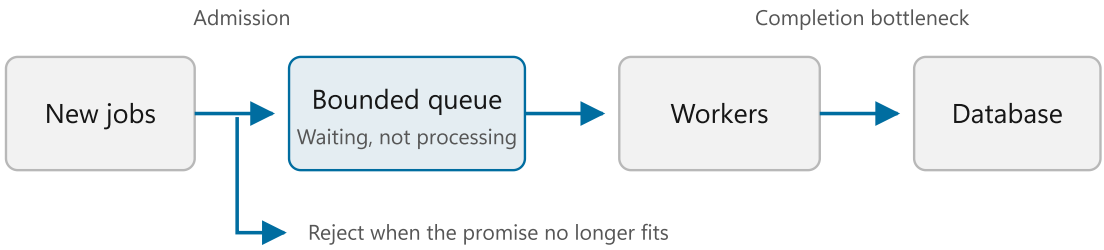
> Why asynchronous work still helps
>
> Separating submission from execution lets a brief burst wait instead of holding client connections open. Durable storage can preserve accepted work while workers restart, provided the service acknowledges only after that storage confirms the write.

These are useful changes to timing and reliability. They do not, by themselves, raise the completion rate of the database or the workers.

01

The queue stores the gap between arrival and completion

A larger waiting room does not make the exit wider.



Workers can take jobs off the queue faster than the database can finish them. Include in-flight work when counting outstanding promises.

02

Recovery needs spare completion capacity

Returning to full utilization is not the same as catching up.

While arrivals exceed completions, unfinished work grows. After the burst, only the completion rate left over after new arrivals can drain that work. If the two rates are equal, the backlog stays.

> Work through a deliberately simple burst

Assume a constant completion capacity of 80 jobs/second. For 30 seconds, 120 jobs/second arrive; afterward arrivals fall to 40 jobs/second. Assume no retries, expiry, or failures, and enough queue space to admit every job.

PHASE	CALCULATION	RESULT
Burst	(120 − 80) × 30	1,200 extra unfinished jobs

PHASE	CALCULATION	RESULT
Recovery headroom	80 − 40	40 jobs/second
Drain time	1,200 ÷ 40	30 seconds after the burst ends

This fluid approximation ignores individual job timing and service-time variation. It estimates aggregate drain time, not a latency percentile or a completion guarantee for each request.

THE HIDDEN COST OF ACCEPTANCE

03

Queue age exposes promises that depth can hide

Watch the oldest unfinished job, not just the number waiting.

The same queue length can represent cheap jobs or slow ones. A nearly empty queue can even hide a stuck job already claimed by a worker. Track age from acceptance through completion, including in-flight work.

> Separate the signals before setting an alert

- Waiting depth describes stored work; in-flight count describes claimed work.
- Oldest unfinished age reveals starvation, blocked workers, and long delays.
- Completion rate and end-to-end latency show what users actually receive.
- Rejection and expiry counts show demand that the service did not satisfy.

Use the report's usefulness deadline to choose an alert budget. An alert should leave time to intervene, not first fire when the oldest accepted report is already useless.

04

Retries spend the capacity recovery needs

A timeout can create more work without canceling the first attempt.

A caller that cannot see progress may submit the same report again. If both attempts run, duplicate work competes with the original backlog. Backoff spreads that demand over time; it does not make the work free.

> RULE
>
> Count attempts against a retry budget, and identify the logical job before admitting another copy.

<details>
<summary>Make deduplication survive a worker restart</summary>

Bind a stable submission key to the intended request and retain its state for the supported retry window. Repeating the same key returns that job's status; reusing it for different input is a conflict, not a new job.

A key at admission does not make later side effects exactly-once. Workers must also make result writes idempotent or commit the result and completion state atomically. Retry only transient failures, with backoff, jitter, and a finite attempt or time budget.
</details>

05

Reject work before its deadline becomes impossible

Bound the waiting time and the storage, not only the worker count.

The service should accept only the work it expects to finish while it is useful. Combine a hard queue/storage cap with an admission estimate that includes waiting, execution, and a safety margin. Reject clearly when either budget is exhausted.

Derive a starting limit, then test its assumptions

For comparable first-in-first-out jobs, dividing the outstanding work ahead by a conservative completion rate gives a rough waiting estimate. Add the new job's execution budget and compare the total with its remaining deadline.

That estimate fails when large jobs dominate, priorities reorder the queue, or the database slows down. Use measured cost classes and age signals to tighten admission, and keep a storage cap even when the estimate looks healthy. A queue limit is a policy backed by tests, not a universal constant.

06

Every accepted job needs a visible ending

Success, expiry, and failure must be distinguishable from waiting.

SUBMISSION

Return a stable job identity only after durable acceptance.

RESOLUTION

Expose completion, terminal failure, or explicit expiry through status.

Separate recovery from endless retry

Before starting stale work, check whether its deadline still allows a useful result. Mark expired jobs explicitly rather than silently discarding them. A permanently failing job needs a terminal state and an owner, not an unlimited return trip through the queue.

A failed-job holding area is an investigation tool, not extra processing capacity. Give retained failures a storage limit and retention policy. Replay only after the cause is understood, through the same admission and retry budgets as new work.

07

Prove the drain path before raising the queue limit

A burst test ends when useful work finishes—not when submissions stop.

Rehearse a burst, a slow dependency, and a worker restart. Keep normal arrivals running during recovery. Check that accepted work reaches a visible ending and that new demand cannot consume the recovery budget unnoticed.

DECISION

Use a bounded queue with explicit admission and job status. Add workers only when the downstream bottleneck has room; increase buffering only when the waiting budget allows it.

> Set acceptance criteria for the rehearsal

1. Choose the report deadline, storage cap, retry budget, and recovery target before the test.
2. Record offered, admitted, completed, expired, and failed work. Include in-flight jobs in reconciliation.
3. Verify the backlog drains within the chosen target while ordinary traffic continues.
4. Repeat with duplicate submissions and a slower database. Check for duplicate effects and terminal jobs stranded as pending.

Do not call an empty queue a success if jobs were merely moved into worker memory, expired without notice, or lost. The evidence is accounted-for outcomes within the chosen contract.

> A queue buys time only when you have the capacity to repay it.